

Manejo de Excepciones

1. Conceptos Básicos de Excepciones

Definición: Una excepción es un evento anómalo que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de las instrucciones. Las excepciones pueden ser provocadas por errores del programa, condiciones del sistema o problemas imprevistos.

Tipos de Excepciones:

- **Excepciones de Hardware:** Errores a nivel de hardware, como división por cero, acceso a memoria inválido.
- **Excepciones de Software:** Errores provocados por el software, como errores de lógica, excepciones definidas por el usuario.
- **Excepciones del Sistema Operativo:** Errores gestionados por el SO, como errores de protección de memoria, interrupciones de E/S.

2. Manejo de Excepciones en Hardware

Proceso:

1. **Detección:** El hardware detecta una condición de excepción, como una operación aritmética inválida.
2. **Interrupción:** El procesador interrumpe la ejecución normal y salta a una rutina de manejo de excepciones predefinida.
3. **Manejo:** La rutina de manejo de excepciones resuelve el problema o notifica al sistema operativo para su manejo.
4. **Reinicio:** Se intenta reanudar la ejecución del programa o se termina si no es posible continuar.

Ejemplo:

- **División por Cero:** El hardware detecta la operación y genera una interrupción que es manejada por una rutina que puede detener el programa y mostrar un mensaje de error.

3. Manejo de Excepciones en Software

Lenguajes de Programación:

- **Try-Catch Blocks (Bloques Try-Catch):** Estructuras que permiten capturar y manejar excepciones.

```
java
Copiar código
try {
    // Código que puede lanzar una excepción
} catch (TipoDeExcepción e) {
    // Código para manejar la excepción
}
```

```
    } finally {  
        // Código que se ejecuta siempre, ocurra o no una excepción  
    }  
}
```

Ejemplo en Java:

```
java  
Copiar código  
try {  
    int resultado = 10 / 0; // Esto lanzará una ArithmeticException  
} catch (ArithmeticException e) {  
    System.out.println("Error: División por cero.");  
} finally {  
    System.out.println("Este bloque se ejecuta siempre.");  
}
```

4. Excepciones y Sistemas Operativos

Función del SO:

- **Interrupciones y Señales:** El SO maneja excepciones mediante interrupciones (hardware) y señales (software), asegurando que el sistema permanezca estable.
- **Protección de Memoria:** El SO utiliza excepciones para manejar violaciones de acceso a memoria, protegiendo áreas críticas del sistema.

Ejemplo:

- **Page Fault:** Cuando un programa accede a una página de memoria que no está presente en la memoria física, el SO maneja esta excepción cargando la página requerida desde el disco.

5. Prácticas de Programación

Buenas Prácticas:

- **Validación de Datos:** Siempre validar los datos de entrada para prevenir excepciones.
- **Manejo de Excepciones:** Capturar y manejar excepciones específicas en lugar de usar excepciones generales.
- **Recursos:** Liberar recursos (memoria, archivos) en un bloque finally para asegurar que se limpian adecuadamente.

Ejemplo en Python:

```
python  
Copiar código  
try:  
    with open('archivo.txt', 'r') as file:  
        data = file.read()  
except FileNotFoundError:  
    print("Archivo no encontrado.")  
finally:  
    print("Operación de archivo finalizada.")
```

6. Seguridad

Importancia:

- **Evitar Explotaciones:** Las excepciones no manejadas pueden ser explotadas para ejecutar código malicioso o provocar fallos en el sistema.
- **Manejo Adecuado:** Asegurarse de que todas las excepciones se manejen adecuadamente y no se filtren detalles sensibles en los mensajes de error.

Ejemplo:

- **Validación de Entradas:** Validar todas las entradas del usuario para evitar inyecciones y otras vulnerabilidades.

```
java
Copiar código
try {
    int numero = Integer.parseInt(userInput);
} catch (NumberFormatException e) {
    System.out.println("Entrada inválida: no es un número.");
}
```